

Session 5 — Study Guide: Node.js & Express

Foundations

This guide explains in depth what changed in B4F Hub in Session 5 and, more importantly, **why**. If you forgot most of the class, you should be able to read this alone and rebuild your understanding. The short version is in the Student Notes; this is the long version.

Before this session began, the Session 4 continuity homework (cross-slice selectors, Save all, and manual `localStorage` persistence for `recentlyViewed.entries`) was reviewed and its official solution taught live, so everyone started today's new content with a fully-working, two-slice, partially-persisted frontend.

1. What is Node.js, actually?

Every line of JavaScript this cohort has written so far ran in exactly one place: a browser tab. That was never the whole truth — it was just the only environment ever used. **JavaScript is a language. The browser is one place that can run it.** Node.js is another: a program, installed on this machine, that executes JavaScript files directly, with no browser involved at all.

Node.js is not a new language — the syntax, the values, the functions are all identical. It is not a framework, either (Express, met later this session, is a framework; Node is what Express runs on top of). The accurate word is **runtime**: a program whose job is to execute JavaScript.

A very concrete, already-familiar example: every time `npm run dev` or `npm run build` has been run this course, that command itself ran *inside* Node.js. `npm` is a Node.js program.

2. Two environments, one language

Each environment adds its own extra APIs on top of the same JavaScript language:

Browser gives code	Node gives code
<code>document</code> , <code>window</code> , <code>localStorage</code>	<code>process</code> , direct file access, network sockets/ports

`console.log` and `JSON.parse` / `JSON.stringify` exist on both sides — they're part of the language itself (or, for `console`, provided independently by each environment), not tied to either one.

This is exactly why `client/src` (React) code cannot run unmodified in Node, and `server/index.js` could not run unmodified in a browser: `document.querySelector` inside `server/index.js` would crash instantly, the same way `process.env` inside a React component would have no meaning.

3. npm, modules, and a process that keeps running

`npm` is Node's package manager — not a React-specific tool. `server/package.json` has its own, independent `dependencies` (just `express`) and is installed/run with the exact same `npm install` / `npm start` commands already familiar from `client/`. `import` / `export` is the same module syntax on both sides, because it's the same language.

A server process is different from a script that runs and exits: once `app.listen(...)` succeeds, the Node process stays alive indefinitely, listening for requests, until it's stopped (Ctrl+C) or crashes. That is why the backend needs its own terminal, separate from the frontend's — both are long-running processes that block whichever terminal runs them in the foreground.

A **port** is the specific "door number" a server listens on: 3001 for this backend, 5173 for Vite's dev server.

4. The real backend, read for the first time

`server/index.js` is not new. It has existed, unmodified, since before Session 1 — `client/src/api.ts` has called it every session. Today's actual new content was reading it, out loud, line by line:

TEXT

```
import express from "express";
import { initialPosts, initialOpportunities } from "../data.js";

const PORT = ...;                                the door number

const app = express();                            creates the app – does NOT start it
app.use(express.json());

let posts = ...
let opportunities = ...

app.get("/api/posts", ...);                        already here since before Session 1
app.post("/api/posts", ...);
app.patch("/api/posts/:id", ...);
app.get("/api/opportunities", ...);
app.patch("/api/opportunities/:id", ...);

app.listen(PORT, () => {                          this line actually starts it
  console.log(`B4F Hub local API running at http://localhost:${PORT}`);
});
```

`express()` only creates the app object — nothing is listening yet. Everything between that line and `app.listen(...)` only *describes* the server. `app.listen(PORT, callback)`, at the very bottom, is the line that actually starts it, binding to port 3001 and running its callback once that succeeds.

5. One real request, traced end to end

Take `fetchOpportunities()`, already called by `OpportunitiesSection.tsx` every time the Opportunities panel loads:

TEXT

```
React component
  ↓
fetch(...)
  ↓
HTTP GET /api/opportunities
  ↓
Vite dev proxy (client/vite.config.ts forwards /api to localhost:3001)
  ↓
Express route
  ↓
handler runs in Node
  ↓
res.json(...)
  ↓
HTTP response
  ↓
React receives JSON
```

The proxy matters: the browser never actually calls `localhost:3001` directly in development — it calls its own origin (`localhost:5173`), and Vite quietly forwards `/api` requests to Express. That is also the reason this project has never needed to deal with CORS: the browser only ever sees one origin, so no cross-origin request ever actually happens.

6. HTTP vocabulary, named on top of the trace

- **Client** — the browser (React's side). **Server** — `server/index.js`, running in Node.
- **Request** — what the client sends. **Response** — what comes back.
- **Method** — `GET` (read, no body) or `POST` (send new data, has a body) matter most today; this app's own `PATCH /api/posts/:id` is one real example of a third method already in use.
- **Path** — `/api/opportunities`, `/api/opportunities/101`, etc.
- **Status code** — `200` success, `201` created (this app's own `POST /api/posts`), `400` the client sent something wrong, `404` nothing exists at that path/id, `500` the server itself failed.
- **JSON body** — plain text shaped like a JavaScript object/array; `res.json(...)` and `JSON.parse` share a name for exactly this reason.

7. `app.get`, `req`, `res` — read fluently

TEXT

```
app.get("/api/example", (req, res) => {
  res.json(...);
});
```

reads as: "when a *GET* request reaches this path, run this function." `req` (request) holds everything about the incoming request. `res` (response) is how the handler answers it — `res.json(data)` (status 200 by default), or `res.status(400).json({ error: "..."} )` — already used by the real `POST /api/posts` handler's own validation, chaining a specific status code onto the same response object before sending JSON.

8. A real, new endpoint: `GET /api/opportunities/:id`

JS

```
app.get("/api/opportunities/:id", (req, res) => {
  const id = Number(req.params.id);
  const opportunity = opportunities.find((candidate) => candidate.id === id);

  if (!opportunity) {
    return res.status(404).json({ error: `No opportunity found with id ${id}.` });
  }

  res.json(opportunity);
});
```

`:id` in the route path is a placeholder Express fills in from the real request path — `req.params.id` reads it back, always as a *string*, which is why `Number(...)` matters. This is not a new pattern: it mirrors the existing `PATCH /api/opportunities/:id` handler right below it, applied to a new method.

Tested directly, before the frontend was ever touched:

BASH

```
curl http://localhost:3001/api/opportunities/101      # 200, the full opportunity
curl -i http://localhost:3001/api/opportunities/99999 # 404, a clear error message
```

9. Reconnecting the frontend

Before this session, visiting any opportunity detail page silently fetched every opportunity and searched for a match client-side — it worked, but it was quietly wasteful. `OpportunityDetailPage` now calls a new function instead:

TS

```
export async function fetchOpportunityById(id: number): Promise<Opportunity | null> {
  const response = await fetch(`${OPPORTUNITIES_URL}/${id}`);
  if (response.status === 404) {
    return null;
  }
  if (!response.ok) {
    throw new Error(await readErrorMessage(response, "Could not load this opportunity."));
  }
  return (await response.json()) as Opportunity;
}
```

A 404 here is an expected, valid outcome ("no opportunity with this id"), not a bug — so it returns `null` instead of throwing, flowing into the exact "not found" branch the detail page already had.

10. `process.env` and a first middleware

JS

```
const PORT = process.env.PORT || 3001;
```

`process` — named back in section 2 as a Node-only API — gets its first real use: the port can now be overridden from outside the file (useful for deployment, or running two copies at once) while `npm start`'s own behaviour stays exactly the same (`|| 3001`).

JS

```
app.use((req, res, next) => {
  console.log(req.method, req.path);
  next();
});
```

This runs for **every** request, before any route, because it's registered with `app.use` (no path) rather than a specific `app.get` / `app.post`. `next()` hands control on to whichever route actually matches; without it, the request would never continue. `app.use(express.json())`, already in the file, is the exact same kind of thing — middleware the class had already been reading past without a name for it.

11. What genuinely did not change

React/Redux stays exactly as Session 4 left it — nothing here is a Redux session. `server/index.js` stays plain JavaScript, not TypeScript. The five pre-existing routes are byte-unchanged. This app still has no database — everything the backend holds lives only in the Node process's own memory, and is lost the moment that process restarts (the same way Redux state was lost on a page reload, before `localStorage` was added in Session 4).

12. What you should be able to explain now

- The one-sentence definition of Node.js, and why "framework" and "language" are both wrong words for it.
- Which real APIs belong to the browser, which belong to Node, and why each side lacks the other's.
- How to read the real `server/index.js` : where Node starts executing, where Express is created, where the port is chosen, where routes live, and which single line actually starts the server.
- The full request/response trace, using this app's own real files, not abstractions.
- `client` , `server` , `request` , `response` , `method` , `path` , `status code` , `JSON body` .
- What `app.get(path, (req, res) => {...})` means, and what `req / res` are for.
- Why `GET /api/opportunities/:id` needed `Number(req.params.id)` , and what its 200/404 decision is based on.
- Why reconnecting the detail page to the new endpoint was a real improvement, not just a rename.
- What `process.env.PORT || 3001` does, and why it changes nothing about today's normal `npm start` .
- What middleware is, what `next()` does, and why forgetting it stalls a request forever.
- Why this app has never needed CORS in development.

13. Not yet

These exist and are **not** part of this session, and will not be needed until later in the course:

- Database integration (MongoDB, PostgreSQL, Mongoose, Prisma) — a dedicated, later block.
- NestJS and Next.js — later, dedicated sessions.
- Authentication (JWT, sessions, cookies, password hashing) — a later, dedicated session.
- Redis, WebSockets, GraphQL — not part of this course's core path this early.
- Redux async patterns (`createAsyncThunk` , `extraReducers` , RTK Query) and React Query — this is not another Redux session; async Redux integration comes later, deliberately.
- A routes/controllers folder split — this session is deliberately about reading a route handler directly, in one file, before any such separation.
- `req.query` , CORS mechanics in depth, deep event-loop/V8 internals — mentioned only if asked, never built.

Glossary

Word	Meaning
Runtime	A program that executes code written in a given language; Node.js executes JavaScript outside the browser.
Client / server	The two ends of an HTTP conversation — the browser and <code>server/index.js</code> , respectively, in this app.
Request / response	What the client sends, and what the server sends back.

Word	Meaning
Method	What kind of HTTP operation a request is — <code>GET</code> (read), <code>POST</code> (create), among others.
Status code	A three-digit number summarising an HTTP response's outcome (200/201/400/404/500 used this session).
<code>req / res</code>	The incoming request and the tool for sending a response, passed to every Express route handler.
<code>req.params</code>	Named segments from a route's own path pattern (<code>:id</code> → <code>req.params.id</code>).
Middleware	A function that runs between a request arriving and a route handler running, registered with <code>app.use</code> , which must call <code>next()</code> to continue.
<code>process.env</code>	Node's own object holding environment variables set outside the JavaScript file itself.
Vite dev proxy	<code>client/vite.config.ts</code> 's <code>server.proxy</code> entry, forwarding <code>/api</code> requests from Vite's origin to Express — the reason this app has never needed CORS.

Review questions

1. Why is "Node.js is a framework" the wrong answer, and what is the correct one-word category?
2. Name one API available in the browser but not in Node, and one available in Node but not in the browser. Why does each environment lack the other's?
3. What is the one line in `server/index.js` that actually starts the server, and why don't the lines above it count?
4. Trace `fetchOpportunities()` from the React component that calls it to the JSON it receives back, naming every real step in between.
5. What does `app.get("/api/opportunities/:id", (req, res) => {...})` mean, read as a sentence?
6. Why does `GET /api/opportunities/:id` need `Number(req.params.id)` specifically?
7. Why was reconnecting `OpportunityDetailPage` to the new endpoint a real improvement, not just a different way of writing the same behaviour?
8. What does `process.env.PORT || 3001` actually do, and what changes about `npm start` 's normal behaviour because of it?
9. Why does the logging middleware need `next()` , and what happens to a request if it's missing?
10. Why has this app never needed CORS in development?